



Software Requirements Specification
Version 1.0

DineIQ Analytics

Project Name: DineIQ Analytics

Theme: MenuMatrix Dining Intelligence

Category: Data Science Intelligence Arena

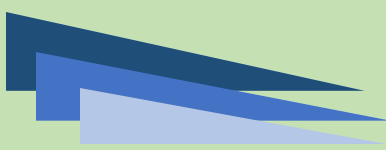


Table of Contents

1.1 Background and Necessity for the Application.....	2
1.2 Proposed Solution.....	3
1.3 Purpose of the Document	25
1.4 Scope of Project.....	25
1.5 Constraints.....	26
1.6 Functional Requirements.....	27
1.7 Non-Functional Requirements.....	33
1.8 Competition Integrity and Anti-Shortcut Requirements	34
1.9 Interface Requirements	36
1.9.1 Hardware	36
1.9.2 Software.....	36
1.10 Project Deliverables	37

1.1 Background and Necessity for the Application

Restaurant businesses generate large volumes of data through customer orders, menu items, pricing, ratings, promotions, inventory consumption, food wastage, ordering channels, and restaurant locations. Analyzing this information is essential for identifying profitable dishes, understanding customer preferences, reducing wastage, improving pricing decisions, and increasing operational efficiency.

Traditional restaurant analysis often depends on basic reports, spreadsheets, or manual comparison of sales figures. These methods may not reveal complex relationships between menu performance, profitability, customer behavior, promotions, ratings, wastage, locations, and time-based ordering patterns. The **DineIQ Analytics** application aims to address this limitation through Big Data processing, Data Science, and predictive analytics.

The '**DineIQ Analytics**' application automatically analyzes large restaurant datasets to identify profitable items, slow-moving dishes, peak ordering periods, and customer segments. It also identifies frequently purchased combinations, wastage patterns, price sensitivity, promotion effectiveness, and unusual transaction behavior. It also predicts future demand and provides data-driven recommendations for menu optimization, inventory planning, pricing, promotions, and customer targeting.

The goal is to deliver a scalable, accurate, and intelligent solution that enables restaurant managers and analysts to make faster and more reliable decisions based on large volumes of data.



1.2 Proposed Solution

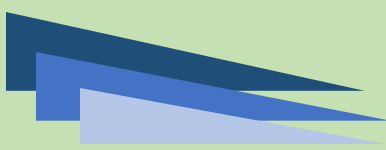
The proposed solution introduces a Big Data and Data Science-powered, Web-based restaurant intelligence platform that analyzes large volumes of restaurant transaction and operational data.

The application collects and processes data related to orders, menu items, customers, restaurant locations, pricing, ratings, promotions, inventory consumption, and food wastage. Apache Spark, PySpark, and Spark SQL are used to process the large datasets, perform transformations, create analytical features, and generate large-scale restaurant insights.

The application analyzes every menu item using multiple indicators such as sales volume, revenue, cost, contribution margin, ratings, repeat purchases, wastage, promotion dependency, and demand trends. Based on these indicators, menu items are classified into categories such as Profit Driver, Volume Driver, Hidden Opportunity, and Low Performer.



The application also performs customer segmentation, market-basket analysis, peak-period analysis, demand forecasting, price sensitivity analysis, promotion effectiveness analysis, wastage prediction, location comparison, and anomaly detection.



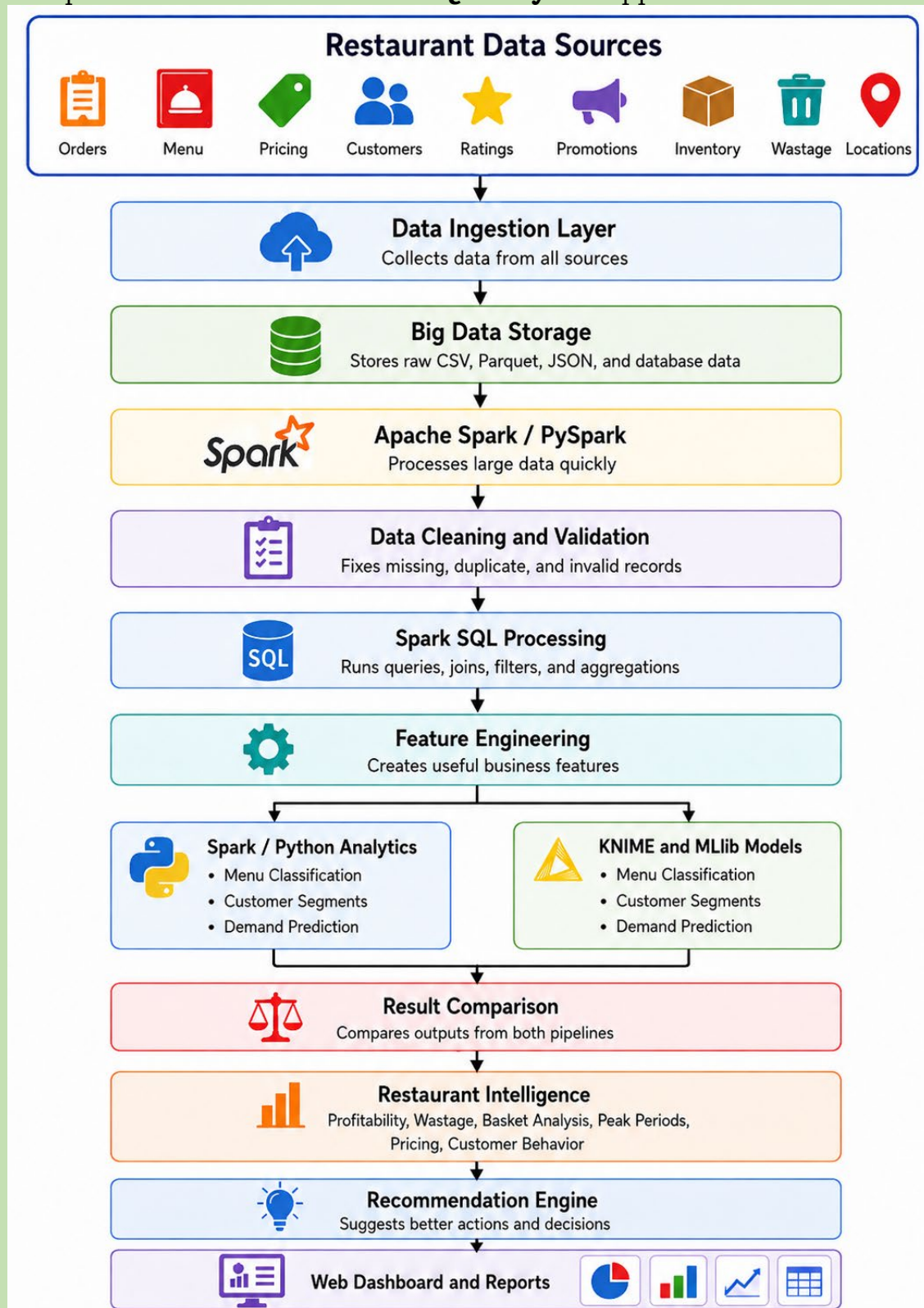
To make the analytical results independently verifiable, teams must develop two separate Data Science pipelines:

1. **Big Data Pipeline:** Apache Spark, PySpark, Spark SQL, and Spark MLlib
2. **Python Data Science Pipeline:** Python, Pandas, Scikit-learn, XGBoost, or other suitable Python Machine Learning libraries

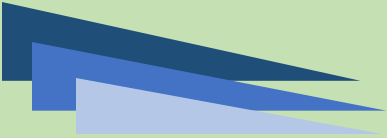
Both pipelines must independently solve selected analytical problems using the same underlying restaurant dataset. The application must compare their predictions or classifications and identify matches, differences, and inconsistencies.

The final Web dashboard must present business insights, predictions, alerts, menu classifications, customer segments, forecasts, and actionable recommendations.

The sample architecture for the **DineIQ Analytics** application is as follows:



Sample Architecture of the Application



Reference Application:

Students may refer to:

<https://incentivio.com/feature/menu-intelligence/>

to understand the general concept of menu intelligence, restaurant analytics, menu-item performance, customer behavior, and data-driven restaurant decision-making.

The reference application must not be copied or reproduced. Teams must design and develop their own original Big Data and Data Science application by implementing all the functional requirements specified in this document.

The competition application must extend beyond basic menu analytics and include:

- Big Data ingestion
- Apache Spark processing
- Multi-table data integration
- Menu profitability intelligence
- Customer segmentation
- Market-basket analysis
- Demand forecasting
- Wastage intelligence
- Pricing intelligence
- Promotion analytics
- Multi-location comparison
- Anomaly detection
- Dual analytical pipelines
- Model-result comparison
- Recommendation engine
- What-if analysis
- Interactive dashboards

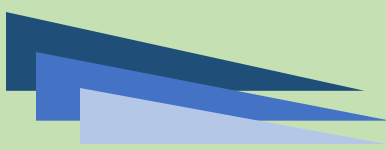
The development phase of the application includes following steps:

Step 1: Restaurant Dataset Creation

Teams must create their own large-scale restaurant dataset containing multiple related tables.

The dataset must contain data related to:

- Customers
- Orders
- Order items
- Menu items
- Menu categories



- Restaurant locations
- Pricing history
- Promotions
- Ratings
- Inventory
- Food wastage
- Ordering channels

Each record must contain appropriate identifiers such as Customer ID, Order ID, Item ID, Location ID, and Promotion ID.

Step 2: Big Data Storage

The generated datasets must be stored in appropriate formats such as:

- CSV
- JSON
- Parquet
- Relational database
- NoSQL database

At least one large, processed dataset must be stored in Parquet format to demonstrate efficient Big Data storage.

Step 3: Data Ingestion Using Apache Spark

Apache Spark or PySpark must be used to load the restaurant datasets.

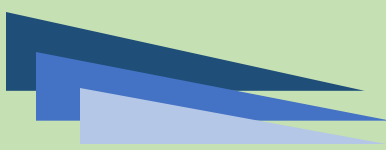
Teams must demonstrate:

- Explicit schema definition
- Schema inference
- Data-type validation
- Large-file loading
- Multiple-file ingestion
- Partition handling

Step 4: Data Quality Assessment

Before analysis, the system must identify:

- Missing values
- Duplicate orders
- Duplicate order-line records



- Invalid menu prices
- Negative quantities
- Invalid dates
- Invalid ratings
- Missing customer IDs
- Missing menu IDs
- Invalid restaurant IDs
- Impossible wastage quantities
- Incorrect discounts
- Cancelled transactions
- Inconsistent units
- Invalid location references

A Data Quality Report must be generated.

Step 5: Data Cleaning

The application must clean, correct, remove, or quarantine problematic records according to documented data-quality rules.

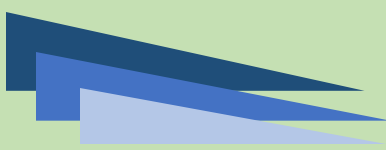
All cleaning decisions must be recorded.

Step 6: Data Integration

The datasets must be integrated using Spark SQL or PySpark joins.

Required relationships should include:

- Orders with customers
- Orders with order items
- Order items with menu items
- Menu items with categories
- Orders with restaurant locations
- Orders with promotions
- Menu items with pricing history
- Menu items with ratings
- Menu items with inventory
- Menu items with wastage



Step 7: Feature Engineering

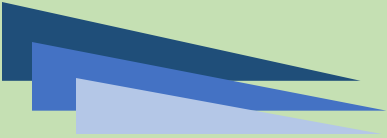
The application must create analytical features such as:

- Item revenue
- Cost
- Contribution margin
- Profit percentage
- Order frequency
- Item popularity
- Repeat-purchase rate
- Average rating
- Rating trend
- Wastage percentage
- Promotion dependency
- Discount percentage
- Customer recency
- Customer frequency
- Customer monetary value
- Average order value
- Peak-hour frequency
- Weekend-order ratio
- Location performance
- Channel preference
- Basket size
- Price-change percentage

Step 8: Exploratory Data Analysis (EDA)

The application must identify:

- Top-selling dishes
- Lowest-selling dishes
- Highest-revenue dishes
- Highest-profit dishes
- Highest-margin dishes
- High-wastage dishes
- Best-rated dishes
- Poorly rated dishes
- Popular menu categories
- Peak ordering periods
- Location-wise sales patterns
- Channel-wise ordering patterns
- Promotion-driven sales



Step 9: Menu Profitability Analysis

The application must calculate menu-item performance using multiple dimensions.

At minimum, analysis must consider:

- Quantity sold
- Revenue
- Ingredients or preparation cost
- Contribution margin
- Profit percentage
- Customer rating
- Repeat-purchase rate
- Wastage percentage
- Promotion dependency
- Sales trend

High sales volume alone must not make a menu item successful.

Step 10: Menu Performance Classification

Each menu item must be classified into one of the following categories:

Profit Driver

High demand and high profitability with acceptable wastage.

Volume Driver

High demand but comparatively lower profitability.

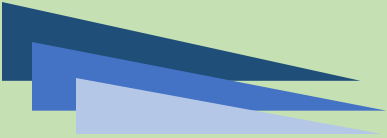
Hidden Opportunity

Good profitability, ratings, or repeat purchase but comparatively low visibility or sales.

Low Performer

Weak demand, weak profitability, excessive wastage, poor ratings, or an unfavorable combination of indicators.

The classification must be data-driven and must not depend on a single hard-coded field.



Step 11: Tricky Menu Performance Cases

The application must correctly handle difficult scenarios such as:

- High-selling, but loss-making dish
- Highly profitable but rarely purchased dish
- Popular dish with excessive wastage
- Highly rated dish with poor profitability
- Low-rated dish with high sales
- Promotion-dependent dish
- Dish performing differently across locations
- Dish performing well only during weekends
- Seasonal menu item
- New menu item with insufficient history

Step 12: Spark MLlib Model Development

Teams must use Spark MLlib to develop predictive or classification models.

At least three suitable algorithms must be trained and compared.

Possible algorithms include:

- Logistic Regression
- Decision Tree
- Random Forest
- Gradient-Boosted Trees
- Linear Regression
- Generalized Linear Regression
- K-Means clustering

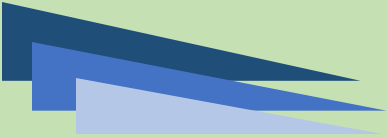
The final model must be selected based on suitable evaluation metrics.

Step 13: Independent Python Data Science Pipeline

A separate Python Data Science pipeline must be developed.

The pipeline may use:

- Pandas
- NumPy
- Scikit-learn
- XGBoost
- SciPy
- Statsmodels
- Other suitable Python libraries



This pipeline must independently process the same underlying restaurant records for selected analytical tasks.

Spark-generated predictions must not simply be exported and reused as Python results.

Step 14: Dual-Pipeline Result Verification

At least one major analytical task must be implemented independently using both:

- Spark MLlib
- Python Data Science

Recommended comparison tasks include:

- Menu performance classification
- Customer segmentation
- Demand prediction
- Wastage-risk prediction
- High-value customer prediction

The outputs must be compared.

The comparison report must include:

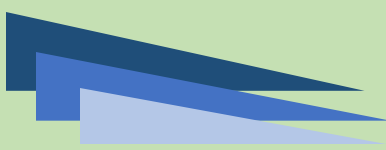
- Record ID
- Actual class or value
- Spark result
- Python result
- Match or mismatch
- Numerical difference, wherever applicable
- Explanation of disagreement
- Overall agreement percentage

Exact equality is not required for independently trained models.

Step 15: Customer Segmentation

Customers must be grouped based on behavioral patterns.
The segmentation should consider:

- Recency
- Frequency
- Monetary value



- Average order value
- Visit frequency
- Favorite menu categories
- Promotion sensitivity
- Ordering channel
- Time-of-day preference
- Repeat behavior

Suggested segments may include:

- High-Value Loyal Customers
- Frequent Customers
- Promotion-Driven Customers
- At-Risk Customers
- New Customers
- Occasional Customers

Step 16: RFM Analysis

For each customer, the application must calculate:

- Recency
- Frequency
- Monetary Value

RFM analysis must be used independently or as input to customer segmentation.

Step 17: Market-Basket Analysis

The application must identify items frequently purchased together.
For suitable item combinations, the application must calculate:

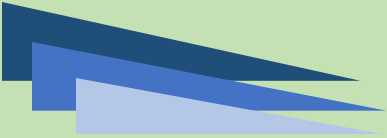
- Support
- Confidence
- Lift

Step 18: Bundle and Cross-Sell Recommendations

Based on market-basket analysis, the application should recommend:

- Combo meals
- Cross-sell opportunities
- Upsell combinations
- Frequently paired dishes

Recommendations must be supported by association-rule evidence.



Step 19: Peak-Period Analysis

The application must identify:

- Peak hours
- Peak days
- Weekend patterns
- Monthly trends
- Seasonal trends
- Location-specific peaks
- Dine-in versus delivery peaks

Step 20: Demand Forecasting

The application must forecast future demand for:

- Menu items
- Menu categories
- Restaurant locations
- Selected time periods

Forecast periods must be configurable.

Step 21: Time-Aware Model Validation

Forecasting and time-based predictive models must not use random data leakage.

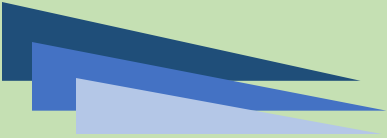
Training data must represent earlier periods and test data must represent later unseen periods.

Future information must not be included in model-training features.

Step 22: Forecast Accuracy Evaluation

Forecasts must be evaluated using suitable measures such as:

- MAE
- RMSE
- MAPE
- R^2 , where appropriate



Step 23: Wastage Analysis

Food wastage must be analyzed by:

- Menu item
- Category
- Location
- Day
- Time period
- Demand
- Inventory consumption
- Promotion
- Preparation quantity

Step 24: Wastage Risk Prediction

The application should predict menu items or periods with a high risk of wastage.

Potential predictive variables may include:

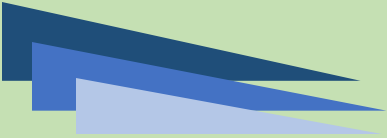
- Historical demand
- Historical wastage
- Day of week
- Season
- Location
- Promotion
- Menu popularity
- Forecast demand
- Preparation quantity

Step 25: Price Intelligence

The application must analyze relationships between:

- Price
- Demand
- Revenue
- Contribution margin
- Discount
- Rating
- Repeat purchase

The application should identify items whose demand changes significantly following price changes.



Step 26: Price-Sensitivity Analysis

The application should classify suitable menu items as:

- Highly Price Sensitive
- Moderately Price Sensitive
- Low Price Sensitivity

Historical price and order behavior should support the result.

Step 27: Promotion Effectiveness Analysis

Promotions must be evaluated using:

- Order volume
- Revenue
- Contribution margin
- Customer acquisition
- Repeat purchases
- Average order value
- Wastage
- Post-promotion behavior

An increase in sales alone must not automatically classify a promotion as successful.

Step 28: Promotion Trap Detection

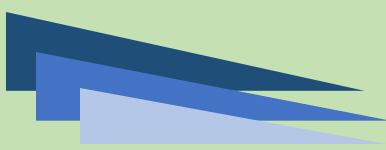
The application must identify misleading promotions such as:

- Sales increase, but profit decreases
- Customer count increases but average margin collapses
- Promotion increases wastage
- Customers purchase only while discounts are active
- Promotion shifts sales away from a more profitable product

Step 29: Rating and Satisfaction Analysis

The application must analyze ratings against:

- Menu items
- Restaurant locations
- Profitability



- Sales
- Repeat purchase
- Time period
- Promotion status

Step 30: Rating Anomaly Detection

The application should flag unusual rating patterns such as:

- Sudden rating spikes
- Sudden rating drops
- Excessive identical ratings
- High number of ratings in a short period
- Ratings inconsistent with purchasing patterns

Step 31: Sales Anomaly Detection

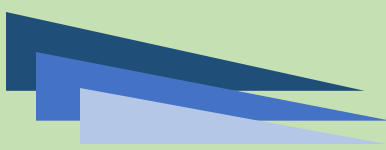
The application must identify unusual events such as:

- Sudden sales spikes
- Sudden sales drops
- Abnormally high order values
- Unusual discounts
- Unexpected demand
- Duplicate transactions

Step 32: Slow-Moving Dish Detection

Slow-moving dishes must be identified using a combination of:

- Low sales volume
- Low purchase frequency
- Long gaps between purchases
- Low repeat purchase
- High wastage
- Weak profitability
- Poor trend



Step 33: Multi-Location Intelligence

Restaurant locations must be compared using:

- Revenue
- Profitability
- Average order value
- Customer count
- Repeat purchase
- Wastage
- Ratings
- Promotion effectiveness
- Menu performance

Step 34: Location-Specific Menu Performance

The same menu item may perform differently at different restaurant locations.

The application must identify location-specific:

- Profit Drivers
- Volume Drivers
- Hidden Opportunities
- Low Performers

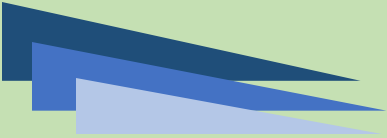
Step 35: Ordering Channel Analysis

The application must analyze ordering behavior across:

- Dine-in
- Takeaway
- Restaurant Website or App
- Third-party delivery platforms
- Other supported channels

The analysis should compare:

- Basket size
- Average order value
- Menu preferences
- Discounts
- Promotions
- Peak periods
- Profitability



Step 36: Customer Churn-Risk Identification

The application should identify customers showing signs of reduced engagement.

Factors may include:

- Increasing recency
- Declining frequency
- Declining monetary value
- Reduced category diversity
- Lower visit frequency

Step 37: Recommendation Engine

The application must generate evidence-based recommendations such as:

- Promote high-margin Hidden Opportunities
- Reduce preparation quantity of high-wastage dishes
- Review pricing of price-sensitive dishes
- Bundle frequently purchased items
- Remove or redesign persistent Low Performers
- Increase stock before predicted peak periods
- Target selected customer segments
- Review ineffective promotions
- Investigate anomalous locations

Step 38: Recommendation Evidence

Every recommendation must display analytical evidence supporting it.

For example:

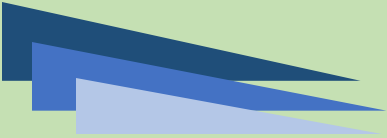
Recommended Action:

- Promote Item M042

Reason:

- High contribution margin
- 4.7 average rating
- Low wastage
- Low current order frequency
- Strong repeat purchase among existing buyers

The application must not provide unexplained recommendations.



Step 39: Recommendation Priority

Recommendations should be assigned a priority such as:

- Low
- Medium
- High
- Critical

Priority should be based on potential business impact.

Step 40: What-If Scenario Analysis

Users must be able to simulate selected scenarios such as:

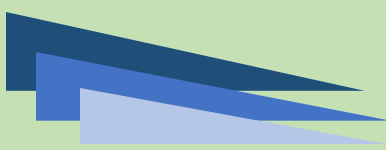
- Increase menu price
- Reduce item price
- Change discount percentage
- Increase promotion frequency
- Remove a menu item
- Reduce preparation quantity
- Increase predicted demand
- Change wastage assumptions

Step 41: Scenario Impact Analysis

The application should display the estimated impact on selected indicators such as:

- Revenue
- Contribution margin
- Demand
- Wastage
- Profitability

The application must clearly identify simulated outputs as estimates rather than actual results.



Step 42: Executive Dashboard

The dashboard should display:

- Total revenue
- Total profit
- Total orders
- Average order value
- Active customers
- Repeat customers
- Wastage
- Forecast demand
- Critical recommendations
- Anomalies

Step 43: Menu Intelligence Dashboard

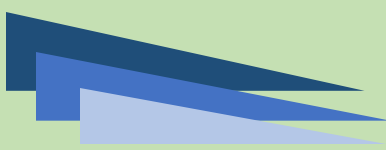
The dashboard must display:

- Menu-item performance
- Profit Drivers
- Volume Drivers
- Hidden Opportunities
- Low Performers
- Slow-moving items
- Ratings
- Margins
- Wastage

Step 44: Customer Intelligence Dashboard

The dashboard should display:

- Customer segments
- RFM distribution
- High-value customers
- At-risk customers
- Promotion-sensitive customers
- Customer trends



Step 45: Wastage Dashboard

The application should display:

- Total wastage
- Wastage cost
- High-wastage items
- High-wastage locations
- Wastage trends
- Wastage-risk predictions

Step 46: Forecast Dashboard

The dashboard should compare:

- Historical demand
- Forecast demand
- Actual versus predicted values
- Forecast errors
- High-risk demand periods

Step 47: Dual-Pipeline Comparison Dashboard

The application should display selected comparisons between Spark and Python models.

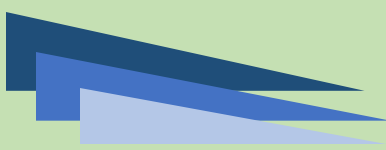
The dashboard should include:

- Spark prediction
- Python prediction
- Match status
- Numerical difference
- Agreement percentage
- Disagreement count

Step 48: Search and Filtering

Users should be able to filter using:

- Date range
- Location
- Menu item
- Menu category
- Customer segment



- Ordering channel
- Promotion
- Performance class
- Price range
- Rating
- Wastage range

Step 49: Downloadable Reports

The application should generate reports for:

- Menu performance
- Profitability
- Customer segmentation
- Market-basket analysis
- Demand forecast
- Wastage
- Promotions
- Pricing
- Location performance
- Anomalies
- Recommendations
- Spark versus Python model comparison

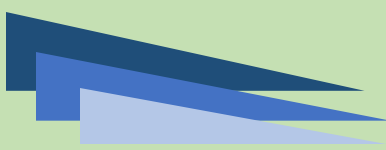
Step 50: Data Export

Users with suitable permissions should be able to export selected analytical results in CSV or Excel-compatible format.

Hint: For this project, teams are expected to create their own realistic and interconnected restaurant dataset. No complete competition-ready dataset will be provided.

The dataset must contain a minimum of:

- **1,000,000 order-line records**
- **At least 100,000 unique orders**
- **At least 50,000 customers**
- **At least 150 menu items**
- **At least 10 menu categories**
- **At least 20 restaurant locations**
- **At least 12 months of transaction history**
- **At least 100,000 rating records**
- **At least 50,000 wastage records**
- Multiple historical pricing records
- Multiple promotion campaigns



The dataset must contain following related tables:

- Customers
- Orders
- Order_Items
- Menu_Items
- Menu_Categories
- Restaurants
- Pricing_History
- Promotions
- Ratings
- Inventory
- Wastage

Relationships must be maintained using appropriate primary and foreign keys.

The data-generation process must create realistic complexity such as:

- Missing values
- Duplicate records
- Invalid transactions
- Cancelled orders
- Changing prices
- Seasonal demand
- Weekend patterns
- Peak-hour patterns
- Multi-location differences
- Promotion periods
- High-value customers
- Churned customers
- New customers
- Popular but low-margin dishes
- Profitable but low-selling dishes
- High-wastage dishes
- Rating anomalies
- Sales anomalies
- Price-sensitive items
- Misleading promotions

Teams must submit the dataset-generation script.

Simply downloading a ready-made restaurant dataset and using it without significant transformation and augmentation will not satisfy the project requirements.

For modelling tasks, teams must divide the dataset appropriately into training, validation, and testing sets.

For time-dependent problems such as demand forecasting, a chronological split must be used to prevent future information from leaking into training data. The Spark and Python pipelines must use equivalent underlying records for comparison.

Predictions generated by Spark must not simply be exported and displayed as Python predictions.



1.3 Purpose of the Document

The purpose of this document is to outline the design, functionality, and implementation plan of the proposed **DineIQ Analytics** application. This document ensures a shared understanding of the project objectives and provides a structured roadmap for the development, testing, and deployment of the application. It is intended for project stakeholders, developers, evaluators, restaurant managers, data analysts, and administrators.

1.4 Scope of Project

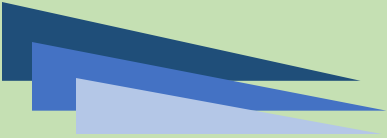
The scope of this project is to design and implement a Big Data and Data Science-powered, Web-based **DineIQ Analytics** application. The application assists restaurant managers, analysts, and administrators in analyzing large volumes of restaurant data. The application will process order, menu, pricing, customer, rating, promotion, inventory, and wastage datasets using Apache Spark, PySpark, Spark SQL, and Machine Learning techniques to identify profitable

items, slow-moving dishes, customer segments, peak periods, ordering patterns, wastage risks, and demand trends. It aims to reduce manual analysis, improve decision-making, support menu and pricing optimization, and generate actionable recommendations based on restaurant performance data. Integration with live POS systems, payment gateways, third-party delivery platforms, or enterprise restaurant management applications falls outside the scope of this project.



1.5 Constraints

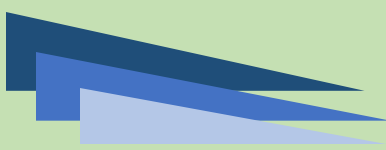
The **DineIQ Analytics** project faces certain constraints such as dependency on quality, completeness, consistency, and accuracy of the order, menu, customer, pricing, rating, promotion, inventory, and wastage datasets. The application's effectiveness is limited by the size, realism, balance, and reliability of the generated dataset as well as the available computational resources for Big Data processing. Differences may also occur between the analytical results or predictions generated by the Apache Spark/PySpark pipeline and the independent Python Data Science pipeline. Privacy and security concerns regarding the storage and handling of customer, transaction, pricing, and restaurant business data must also be addressed in accordance with applicable data protection regulations.



1.6 Functional Requirements

Creating a **DineIQ Analytics** application involves several steps and components. Following are the functional requirements:

- i. **User Registration and Authentication** - The application should allow restaurant managers, analysts, regional managers, and administrators to securely log in.
- ii. **Role Based Access Control** - The application should provide permissions according to user role.
- iii. **Restaurant Location Management** - Administrators should be able to create and maintain restaurant-location records.
- iv. **Menu Management** - The application should store menu categories, items, prices, costs, descriptions, and availability.
- v. **Pricing History Management** - Historical price changes should be maintained for every menu item.
- vi. **Customer Data Management** - The application should maintain anonymized customer profiles and behavioral information.
- vii. **Order Management** - The application should store order headers and detailed order-line transactions.
- viii. **Promotion Management** - Promotions, discounts, coupons, campaign periods, and applicable menu items should be maintained.
- ix. **Rating Management** - Customer rating information should be stored and linked to relevant menu items or restaurant locations.
- x. **Inventory Management** - Inventory usage, stock, replenishment, and item consumption should be maintained.
- xi. **Wastage Management** - Wastage records should include quantity, cost, item, location, date, and reason.



xii. Big Data Ingestion - Spark/PySpark should be used to ingest large restaurant datasets.

xiii. Schema Validation - Required columns, data types, and relationships should be validated.



xiv. Data Quality Analysis - Missing, duplicate, inconsistent, invalid, and anomalous records should be identified.

xv. Data Cleaning - Data problems should be corrected or managed according to documented rules.

xvi. Spark SQL Processing - Spark SQL should be used for complex analytical queries and aggregation.

xvii. Data Partitioning - Large datasets should be appropriately partitioned.

xviii. Parquet Storage - At least one processed Big Data dataset should be stored in Parquet format.

xix. Feature - Derived analytical and Machine Learning features should be generated.



xx. Profitability Analysis - Revenue, cost, contribution margin, and profitability should be calculated.

xxi. Menu Performance Classification - Every menu item should be classified into defined performance groups.

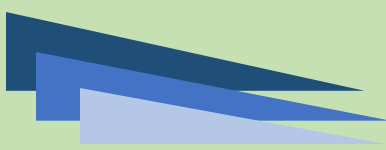
xxii. Peak-Period Detection - Peak ordering hours, days, and seasonal periods should be identified.

xxiii. Customer Segmentation - Customers should be segmented according to behavioral patterns.

xxiv. RFM Analysis - Recency, Frequency, and Monetary Value should be calculated.



xxv. Market-Basket Analysis - Frequently purchased menu combinations should be discovered.



xxvi. Association-Rule Metrics - Support, confidence, and lift should be calculated.

xxvii. Bundle Recommendation - Frequently associated products should be recommended for bundling or cross-selling.

xxviii. Demand Forecasting - Future demand should be predicted.

xxix. Forecast Evaluation - Forecasts should be evaluated using appropriate metrics.

xxx. Wastage Analysis - Food wastage trends should be analyzed.



xxxi. Wastage Prediction - High-risk wastage items or periods should be predicted.

xxxii. Price-Sensitivity Analysis - Changes in demand following price changes should be analyzed.



xxxiii. Promotion Effectiveness Analysis - Promotions should be evaluated using revenue, profit, customer behavior, and wastage.

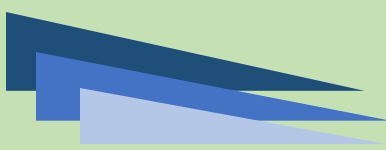
xxxiv. Promotion Trap Detection - Promotions that increase sales, but damage profitability or other KPIs should be identified.

xxxv. Rating Analysis - Customer ratings should be analyzed against item and restaurant performance.

xxxvi. Rating Anomaly Detection - Unusual rating patterns should be flagged.

xxxvii. Sales Anomaly Detection - Unusual sales and transaction behavior should be detected.

xxxviii. Location Comparison - Restaurant locations should be compared using standardized performance indicators.



xxxix. Location-Specific Menu Intelligence - Menu items should be analyzed independently across locations.



xl. Ordering Chanel Analysis - Dine-in, takeaway, Website/App, and delivery behavior should be compared.

xli. Customer Churn-Risk Analysis - Customers showing declining engagement should be identified.

xlii. Spark MLib Model Development - At least three suitable Spark models should be trained and evaluated.

xliii. Independent Python Model Development - Separate Python Machine Learning models should be developed.



xliv. Dual-Pipeline Prediction Comparison - Spark and Python predictions should be compared.

xlv. Model Competency Analysis - Agreement and disagreement between models should be calculated.

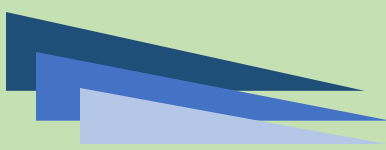


xlvi. Model Evaluation - Suitable Machine Learning evaluation metrics should be provided.

xlvii. Recommendation Engine - The application should generate evidence-based recommendations.

xlviii. Menu Optimization Recommendations - Items should be recommended for promotion, repricing, bundling, redesign, or removal.

xl ix. Inventory Recommendations - Forecast and wastage results should support inventory planning.



1. **Customer Targeting Recommendations** - Customer segments should be mapped to suitable strategies.

- li. **What-If Analysis** - Users should be able to simulate selected pricing, promotion, demand, and inventory changes.

- lii. **Executive Dashboard** - Important restaurant KPIs should be displayed.

- liii. **Menu Dashboard** - Menu-item performance should be displayed.

- liv. **Customer Dashboard** - Customer segments and behavior should be displayed.

- lv. **Wastage Dashboard** - Wastage records should include quantity, cost, item, location, date, and reason.

- lvi. **Forecast Dashboard** - Historical and forecast demand should be displayed.

- lvii. **Dual-Pipeline Dashboard** - Spark and Python results should be compared.

- lviii. **Search and Filtering** - Analytics should support multiple filtering options.

- lix. **Downloadable Reports** - Analytical reports should be downloadable

- lx. **Data Export** - Selected results should be exportable.

- lxi. **Database Storage** - Application configuration, metadata, users, recommendations, and results should be securely stored.

- lxii. **Model Version Tracking** - Predictions should identify the model version used.





lxiii. Audit Trail - Data-processing jobs, predictions, exports, and administrative actions should be logged.

lxiv. Error Handling - Understandable errors should be displayed for processing, model, Spark, and database failures.

lxv. Spark Job Monitoring - Data-processing and job status should be displayed.

lxvi. Responsive Web Interface - The application should provide an intuitive Web interface for supported browsers.

In addition, the application must provide insights rather than only visualizations. Dashboards that merely display charts without analytical conclusions, predictive models, recommendations, and Big Data processing will not satisfy the project requirements.



1.7 Non-Functional Requirements

There are several non-functional requirements that should be fulfilled by the application which are as follows:



1. **Performance:** The application should process uploaded claim details and generate predictions from both models within five seconds under normal operating conditions.
2. **Scalable:** The architecture should support increasing the dataset to at least five million order-line records without requiring a complete redesign.
3. **Usable:** The application should provide an intuitive and user-friendly Web interface for restaurant managers, analysts, regional managers, and administrators.

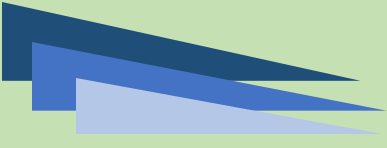


4. **Accuracy:** Classification models should target at least 85% test accuracy or a macro F1-score of at least 0.80 where appropriate. Forecasting models must demonstrate measurable improvement over a simple baseline forecast.
5. **Available:** The application should remain operational during required evaluation periods with at least 99% uptime under normal operating conditions.



These are the bare minimum expectations from the project. It is a must to implement the FUNCTIONAL and NON-FUNCTIONAL requirements given in this SRS.

Once they are complete, you can use your own creativity and imagination to add more features if required.



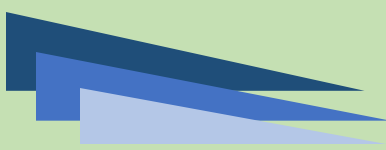
1.8 Competition Integrity and Anti-Shortcut Requirements

To ensure that the project represents genuine team effort, the following requirements are mandatory:

1. Each team member must explain the modules assigned to them.
2. The GitHub repository must contain meaningful commits across all five competition days.
3. Teams must maintain a development log showing:
 - Work completed
 - Dataset changes
 - Data-quality problems
 - Spark failures
 - Model failures
 - Changes made
 - Tests performed
 - Performance improvements
4. Evaluators may select any Spark transformation, Spark SQL query, Python function, Machine Learning model, feature, or recommendation and ask the team to explain it.
5. Teams must complete a surprise modification during the final evaluation, such as:
 - Adding a restaurant location
 - Adding a new menu category
 - Changing profitability thresholds
 - Changing a feature
 - Adding a Spark transformation
 - Modifying the forecast window
 - Adding a new KPI
 - Adding an anomaly rule
 - Adding a new dashboard filter
6. Teams may be required to fix a deliberately introduced defect.
7. Unused code, unexplained code, fabricated metrics, hard-coded insights, or hidden external decision APIs may result in disqualification.
8. The application will be tested using a hidden restaurant dataset unavailable before final evaluation.
9. Hidden datasets may contain:
 - Missing values
 - Duplicate orders
 - Unknown menu items
 - New restaurant locations
 - Price changes
 - Unusual promotions
 - Extreme wastage
 - Seasonal changes

- Unexpected customer behavior
 - Outliers
- Teams must demonstrate that Spark and Python models generate their results independently.
 - Spark predictions must not be copied into the Python pipeline or vice versa.
 - Teams may be asked to explain why the two models disagree.
 - Any AI tool used during development must be declared in an AI_USAGE.md file.
 - The declaration must include:
 - Tool name
 - Purpose
 - Files or modules affected
 - Changes made
 - Testing performed
 - AI-generated code must be independently reviewed, modified where required, understood, and tested by team members.
 - The final restaurant analytics, predictions, and recommendations must not be generated through an external generative-AI decision API.
 - Failure to explain or modify submitted code may result in reduced or zero marks for the affected module.





1.9 Interface Requirements

1.9.1 Hardware

Intel Core i5/i7 Processor or higher
8/16 GB RAM or higher recommended
Colour SVGA monitor
500 GB or more free disk space
Mouse and Keyboard
Stable Internet connection
For larger local Spark workloads, systems with higher RAM and multi-core processors are recommended.

1.9.2 Software

Technologies to be used:

1. **Frontend:** HTML5, CSS3, JavaScript, Bootstrap, Streamlit, React, or any other suitable frontend technology
2. **Backend:** Flask, Django, FastAPI, or Streamlit
3. **Big Data Processing:** Apache Spark, PySpark, Spark SQL
4. **Data Store:** CSV, JSON, Parquet, or Hadoop-compatible storage
5. **Database:** MongoDB, PostgreSQL, MySQL, Firebase, SQLite, or another suitable relational or NoSQL database
6. **Programming/IDE:** Python, Jupyter Notebook, Anaconda, Google Colab, PyCharm, or Visual Studio Code
7. **Data Science and Analytics:** Apache Spark MLlib, Scikit-learn, XGBoost, Statsmodels, or other suitable Machine Learning libraries
8. **Data Science Libraries:** NumPy, Pandas, SciPy, Matplotlib, Plotly, Joblib, and other suitable libraries
9. **Visualization:** Plotly, Matplotlib, Streamlit charts, Power BI, Tableau, or suitable Web-based visualization tools
10. **Testing:** pytest, unittest, Postman, Spark testing utilities, or other suitable tools
11. **Version Control:** Git and GitHub
12. **Deployment:** Render, Railway, PythonAnywhere, Streamlit Community Cloud, or another suitable hosting platform



1.10 Project Deliverables

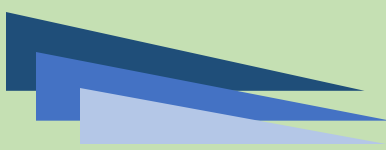
Each team must design, build, test, document, deploy, and demonstrate the complete **DineIQ Analytics** application.

The final submission must contain the following items:

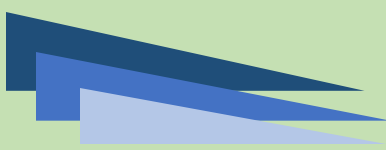
1. Project Report

The project report must include:

- Problem definition
- Background and business necessity
- Proposed solution
- Purpose of the document
- Scope of project
- Assumptions
- Constraints
- Functional requirements
- Non-functional requirements



- System architecture
- Big Data architecture
- Module descriptions
- Database design
- Entity relationship design
- Data dictionary
- Data Flow Diagram
- Use Case Diagram
- Activity Diagram
- Sequence Diagram
- Analytical workflow
- Dataset-generation methodology
- Data-quality methodology
- Spark-processing pipeline
- Spark SQL processing
- Feature engineering
- Menu classification methodology
- Customer segmentation
- Market-basket analysis
- Demand forecasting
- Wastage analysis
- Pricing analysis
- Promotion analysis
- Anomaly detection
- Spark MLlib model design
- Python Data Science model design
- Dual-pipeline result comparison
- Model evaluation



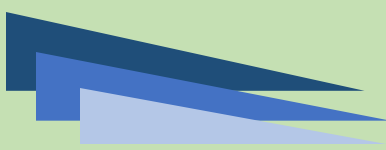
- Testing strategy
- Security considerations
- Privacy considerations
- Limitations
- Future enhancements

2. Source Code

The complete source code must be uploaded to a public GitHub repository.

The repository should include:

```
README.md
AI_USAGE.md
requirements.txt
LICENSE
src/
templates/
static/
data_generator/
raw_data/
processed_data/
parquet_data/
spark_jobs/
spark_sql/
python_pipeline/
notebooks/
models/
database/
tests/
sample_data/
```



documentation/

screenshots/

reports/

config/

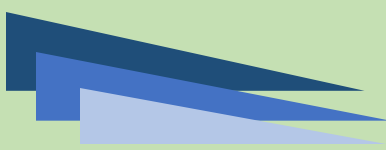
• The source-code submission must include:

- Frontend files
- Backend files
- Dataset-generation scripts
- Data-quality scripts
- Spark ingestion scripts
- Spark transformation scripts
- Spark SQL scripts
- Feature-engineering scripts
- Spark MLib model scripts
- Python Data Science scripts
- Forecasting scripts
- Basket-analysis scripts
- Wastage-analysis scripts
- Recommendation-engine scripts
- Saved model files
- Automated tests

3. Big Data Dataset

The submission must include:

- Dataset-generation scripts
- Data dictionary
- Dataset schemas
- Primary and foreign key definitions
- Dataset statistics
- Raw sample data



- Cleaned sample data
- Parquet datasets
- Training dataset
- Validation dataset
- Testing dataset
- Hidden-data readiness documentation

If the complete dataset cannot be stored on GitHub because of file-size limitations, it may be stored using an appropriate external storage service. Complete public access instructions must be provided.

4. Spark Processing Evidence

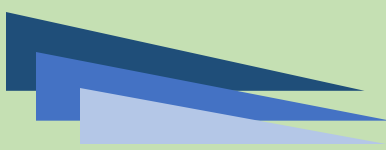
Teams must submit:

- Spark ingestion scripts
- Schema definitions
- Data-quality results
- Cleaning transformations
- Join operations
- Spark SQL queries
- Aggregation operations
- Partition strategy
- Parquet output
- Spark execution logs
- Processing-time results
- Data-quality reports

5. Spark MLlib Evidence

Teams must submit:

- Features used
- Algorithms tested
- Hyperparameters
- Training results
- Validation results
- Testing results



- Evaluation metrics
- Confusion matrix, where applicable
- Saved Spark model
- Model version
- Sample predictions

At least three suitable models must be compared before selecting the final model.

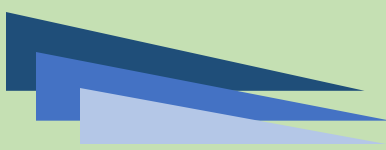
6. Python Data Science Model Evidence

Teams must submit:

- Python preprocessing scripts
- Feature-engineering scripts
- Algorithms tested
- Hyperparameter settings
- Training results
- Validation results
- Testing results
- Confusion matrix, wherever applicable
- Accuracy
- Precision
- Recall
- F1-score
- Forecast metrics, wherever applicable
- Saved Python model
- Model version
- Sample predictions

7. Dual-Pipeline Model Comparison Report

A separate report must compare Spark and Python results on at least **100 unseen records or analytical cases**.



The report must include:

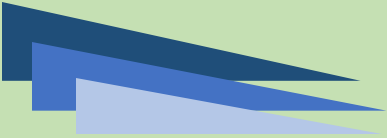
- Record or Entity ID
- Actual class or value
- Spark prediction or result
- Python prediction or result
- Match or mismatch
- Numerical difference, wherever applicable
- Model confidence or probability, where supported
- Final consistency status
- Explanation of major disagreements
- Overall agreement percentage

Teams must independently generate both results.

8. Restaurant Intelligence Report

The report must include:

- Most profitable dishes
- Highest-volume dishes
- Hidden Opportunities
- Low Performers
- Slow-moving dishes
- High-wastage dishes
- Peak ordering periods
- Customer segments
- High-value customers
- Churn-risk customers
- Popular combinations
- Demand forecasts
- Promotion effectiveness
- Price-sensitive items

- 
- Location performance
 - Anomalies
 - Final recommendations

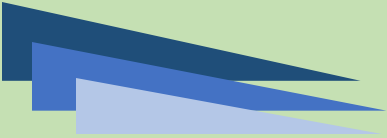
9. Test Cases

The submission must include:

- Functional tests
- Integration tests
- Big Data ingestion tests
- Schema validation tests
- Data-quality tests
- Spark transformation tests
- Spark SQL tests
- Spark model tests
- Python model tests
- Dual-pipeline comparison tests
- Forecast tests
- Basket-analysis tests
- Wastage tests
- Promotion tests
- Pricing tests
- Anomaly tests
- Security tests
- Boundary tests
- Hidden-data readiness tests

The project must demonstrate difficult cases including:

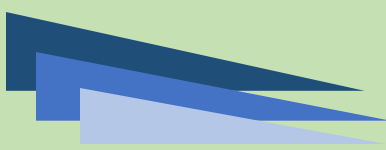
- High-selling loss-making dish
- Low-selling high-margin dish
- High-wastage popular dish

- 
- Promotion increasing sales but reducing profit
 - Dish performing differently across locations
 - New menu item
 - Price-sensitive item
 - Customer churn
 - Rating anomaly
 - Sales anomaly
 - Spark/Python disagreement case

10. Installation Instructions

Documentation must provide:

- Supported operating system
- Python version
- Java installation
- Apache Spark installation
- PySpark configuration
- Virtual environment creation
- Python dependency installation
- Database setup
- Dataset-generation steps
- Parquet-generation steps
- Spark execution steps
- Python pipeline execution steps
- Web application execution
- Test execution
- Troubleshooting



11. Execution Instructions

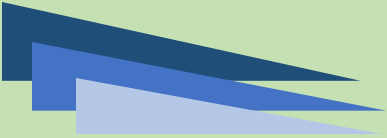
The `README.md` file must explain:

- How to install the project
- How to log in
- How to generate the dataset
- How to load the dataset
- How to run Spark processing
- How to execute Spark SQL
- How to train Spark models
- How to train Python models
- How to compare model results
- How to perform menu analysis
- How to view customer segments
- How to perform basket analysis
- How to generate forecasts
- How to analyze wastage
- How to perform what-if analysis
- How to view recommendations
- How to access dashboards
- How to export reports
- How to execute automated tests

12. GitHub Repository

The GitHub repository must:

- Have public access
- Contain meaningful commits
- Include commits from every team member
- Include complete source code
- Include data-generation scripts

- 
- Include Spark scripts
 - Include Spark SQL queries
 - Include Python Data Science pipeline
 - Include model files or model-generation instructions
 - Include sample data
 - Include screenshots
 - Include evaluator credentials
 - Include test results
 - Include assumptions
 - Include limitations
 - Include project report or report link
 - Include technical blog link
 - Include demonstration video link

The public GitHub URL must be submitted.

13. Deployed Application

Teams should submit:

- Public Web application URL
- Evaluator login credentials
- Administrator login credentials
- Sample analytical data
- Instructions for testing the application

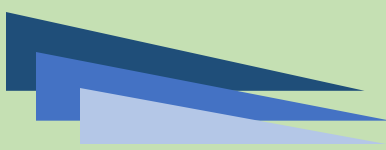
Where public deployment is not possible, complete local installation and execution instructions must be provided.

14. Demonstration Video

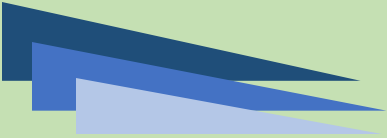
A demonstration video in .mp4 format is mandatory.

The video must demonstrate:

- Login
- Dataset generation



- Big Data ingestion
 - Spark processing
 - Data-quality analysis
 - Data cleaning
 - Spark SQL queries
 - Feature engineering
 - Menu profitability
 - Menu classification
 - Customer segmentation
 - RFM analysis
 - Market-basket analysis
 - Peak-period analysis
 - Demand forecasting
 - Wastage analysis
 - Price intelligence
 - Promotion analysis
 - Anomaly detection
 - Spark model prediction
 - Python model prediction
 - Dual-pipeline comparison
 - Recommendation engine
 - What-if scenario
 - Dashboard
 - Report generation
 - At least one difficult contradictory business case
- Submission of the demonstration video is mandatory.

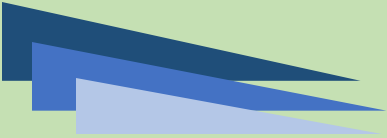


15. Technical Blog

Each team must publish a technical blog of at least 2,000 words on Blogger, Tumblr, Ghost, Medium, or another suitable free blogging platform.

The blog must discuss:

- Business problem
- Background and necessity
- Proposed solution
- Big Data architecture
- Dataset generation
- Dataset complexity
- Data quality
- Apache Spark
- PySpark
- Spark SQL
- Parquet
- Feature engineering
- Menu intelligence
- Customer segmentation
- RFM analysis
- Market-basket analysis
- Demand forecasting
- Wastage prediction
- Pricing intelligence
- Promotion intelligence
- Spark MLlib
- Python Data Science pipeline
- Model comparison
- Analytical disagreements

- 
- Challenges
 - Performance
 - Testing
 - Security
 - Lessons learned
 - Limitations
 - Future enhancements

The published blog link must be included in the project documentation and GitHub repository.

16. AI Tool Usage Declaration

Teams must not directly submit content or code generated by GPTs or other AI tools without independently reviewing, modifying, testing, and understanding it.

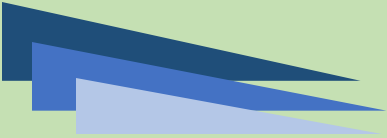
Any AI tool used during development must be declared in `AI_USAGE.md`.

The declaration must include:

- Name of the AI tool
- Purpose of use
- Type of assistance requested
- Files or modules affected
- Modifications performed
- Testing completed
- Name of the team member who verified the output

AI-generated images may be used for visual representation, but the AI tool used must be declared.

The final restaurant analytics, predictions, classifications, recommendations, and forecasts must be produced by the team's own Spark, Python, Data Science, Machine Learning, and application logic.



17. Final Submission Checklist

The final submission must include:

- Project report
- **Public GitHub repository URL**
- Complete source code
- Dataset-generation scripts
- Big Data dataset
- Data dictionary
- Spark jobs
- Spark SQL scripts
- Parquet datasets
- Spark MLib models
- Python Data Science models
- Dual-pipeline comparison report
- Restaurant Intelligence report
- Test cases and results
- Installation instructions
- Execution instructions
- Deployment URL, wherever applicable
- Demonstration video
- Technical blog
- Project presentation
- AI_USAGE.md
- Team contribution record

Documentation is a very important part of the project. All important aspects of the dataset, Big Data architecture, Spark processing, Data Science pipelines, models, forecasts, recommendations, testing, installation, execution, security, and limitations must be documented completely and clearly.

~~~ End of Document ~~~